Part 1: Schema Design and Sharding

1. My proposed Row Key for this dataset is a geographically partitioned one with the structure *<region_code>#<hash_bucket>#<DOL_vehicle_ID>*, with the region code identifying a vehicle's county and the hash bucket being calculated from its DOL Vehicle ID. An issue I noticed is how unevenly the dataset distributed vehicles across locations. Seattle itself had 45000+ records, which made me hesitant to use a row key organized by location only. Dividing each region into hash buckets would reduce the chance of localized write hotspotting as all of its records wouldn't be stored under one uninterrupted key prefix. Additionally, county-level queries could scan the buckets grouped with that county. City and Make searches could utilize two lookup tables, *city_locator* and *make_locator*, which have references to corresponding primary vehicle rows. I would keep separate paths for these searches.
2. I would design the Column Families by the attributes' role in answering different queries.

Family	**Attributes**
catalog	VIN, Make, Model, Model Year
geography	Vehicle Location, 2020 GEOID, Postal Code, City, County, State
energy_profile	Electric Vehicle Type, Electric Range, CAFV eligibility
grid_context	Electric Utility, Legislative District

With this grouping, frequently accessed vehicle attributes can be retrieved without also gathering less frequently needed utility data, reducing unnecessary data transfer. Bigtable can filter reads by column family, thereby reducing unneeded data transfer. For example, a user examining where longer-range EVs are registered and which utilities serve those areas would have little reason to retrieve every vehicle's model information.

3. A hashed key based on DOL Vehicle ID would separate records fairly evenly since placement wouldn't be dependent on popular manufacturers or model years. When the vehicle ID is known, direct lookups are also efficient. Grouped searches, however, would be more expensive since records sharing the same manufacturer, for example, would be scattered across the database in different parts. A range-based key like *<Model_Year>#<Make>#<DOL_vehicle_ID>* would retain chronological organization and make model-year comparisons easier to make. But, if writes repeatedly target the same model-year range, traffic could become concentrated in a narrow part of the key space, potentially creating a hotspotting issue. Overall, I would prefer the hashed approach for balancing cluster workloads, and a range-based approach when most queries involve consecutive model years.

Part 4: Performance Analysis

I would use primary-only writes for a system collecting EV battery-health readings during periodic inspections. Not every reading has to become a permanent record since abnormal measurements can be flagged for a second inspection. I believe the value of these early readings is in identifying potential problems. In this case, I would prioritize the fast processing of incoming readings, while reserving stronger write guarantees for finalized diagnostic results.

In my experiment, I sent 500 requests to each endpoint from my Mac. All 1,000 succeeded. With w=1, the primary-only endpoint */insert-fast* averaged 48.63 ms, while the majority-acknowledged endpoint */insert-safe* averaged 63.90 ms. This leaves a 15.27 ms difference between both approaches.

The CAP theorem becomes relevant when a network partition prevents database nodes from communicating normally. A primary-only acknowledgement leaves a window where an accepted reading could disappear before reaching other replicas. I would accept that possibility for initial battery measurements that can be easily remeasured, but I wouldn't for finalized diagnostic reports, where losing a confirmed result is higher risk.